UNIVERSITÀ DEGLI STUDI DI NAPOLI FEDERICO II

Scuola Politecnica e delle Scienze di Base
Corso di Laurea Triennale in Informatica

Insegnamento: Laboratorio di Sistemi Operativi (Esame da 6 CFU)

RELAZIONE TECNICA E SCELTE IMPLEMENTATIVE

Progetto 2: Battaglia Navale Concorrente

Architettura Client-Server in C con Socket TCP e Thread POSIX

Componenti del Gruppo:	**Matricola:**
Carrino Emanuele	**N86005084**
Ascione Nicola	**N86005133**

Anno Accademico 2025/2026

1. Introduzione ed Obiettivi del Progetto

Il presente elaborato descrive l'architettura software, le decisioni progettuali e le soluzioni algoritmiche adottate per la realizzazione di un sistema distribuito per il gioco della **Battaglia Navale** in ambiente di rete, conforme ai requisiti dell'esame di *Laboratorio di Sistemi Operativi* (ordinamento da 6 CFU).

In conformità ai vincoli per l'ordinamento da 6 CFU, l'intero sistema è implementato in linguaggio **C standard (C11 / POSIX.1-2008)** sia per la componente Server che per la componente Client, senza ricorrere a container Docker e basandosi rigorosamente sulle primitive fornite dal sistema operativo Linux.

Gli obiettivi prioritari perseguiti durante la progettazione sono stati:

- **Autoritarietà del Server (Server-Authoritative):** la logica di gioco, lo stato delle navi e la validità delle mosse risiedono esclusivamente sul Server. Il Client funge da interfaccia di presentazione, prevenendo incoerenze o manomissioni delle griglie.
- **Concorrenza Parallela:** gestione contemporanea di molteplici giocatori e sessioni di gioco indipendenti mediante Thread POSIX nativi (pthread).
- **Robustezza Estrema (Hardening Difensivo):** blindatura del codice contro input utente o di rete errati o malformati (fuzzing), prevenendo crash, memory leak o loop infiniti.
- **Pulizia ed Eleganza Architetturale:** codice lineare, modulare e conforme agli standard UNIX/POSIX per l'elaborazione di sistema.

2. Architettura di Rete e Comunicazione Socket TCP

La comunicazione tra i processi Client e Server avviene mediante la **Socket API POSIX** su protocollo TCP. La scelta di TCP rispetto a UDP è motivata dalla natura orientata alla connessione, sequenziale e affidabile del protocollo (SOCK_STREAM), essenziale per uno scambio deterministico di comandi a turni.

Dettagli di configurazione e primitive utilizzate:

- **Lato Server:** viene creato un endpoint con socket(AF_INET, SOCK_STREAM, 0). Per evitare l'errore *'Address already in use'* in caso di riavvio rapido del processo, viene abilitata l'opzione SO_REUSEADDR tramite setsockopt(). L'indirizzo locale viene configurato impostando sin_family = AF_INET, sin_port = htons(PORT) e sin_addr.s_addr = htonl(INADDR_ANY), per poi invocare bind() e porre il descrittore in ascolto con listen(server_fd, 16).
- **Lato Client:** per specificare l'indirizzo del server si impiega la funzione standard POSIX inet_aton(), che converte una stringa IPv4 in formato dotted nella corrispondente struttura binaria di rete. La connessione viene stabilita mediante connect().

> ***Approfondimento Tecnico sulle Socket POSIX:***
> *«Il socket definisce un canale di comunicazione bidirezionale a flusso continuo. SOCK_STREAM garantisce integrità, ordine di consegna e assenza di duplicazioni dei byte. Il server chiama bind per associare l'indirizzo al descrittore, listen per predisporre la coda delle richieste e accept per creare il canale dedicato a ciascun client connesso.»*

3. Modello di Concorrenza: Server Multithreaded

Per gestire più client in parallelo (requisito vincolante della traccia), il server implementa il modello **Multithreaded**.

Confronto Thread POSIX vs Processi (fork):
La duplicazione di processi tramite fork() genera spazi di indirizzamento separati e non condivisi. In un'applicazione come la Battaglia Navale, dove due client interagiscono sulla medesima partita, l'uso di fork() avrebbe richiesto complessi meccanismi di memoria condivisa POSIX (shm_open, mmap) e semafori IPC di

sistema. I **Thread POSIX (`pthread`)** condividono naturalmente la memoria del processo padre, riducendo l'overhead di context-switch e consentendo una sincronizzazione rapida ed efficiente tramite Mutex in memoria.

Gestione della Terminazione con `pthread_detach()`:
I thread terminati per cui non viene invocata pthread_join() rimangono residenti in memoria nello stato di *'zombie'*. Per scongiurare questo consumo di risorse senza costringere il thread principale a chiamate bloccanti, all'avvio della funzione worker viene eseguita la chiamata:
`pthread_detach(pthread_self());`
In questo modo il sistema operativo dealloca automaticamente lo stack e i descrittori del thread al momento della sua uscita.

4. Sincronizzazione tra Thread e Prevenzione dei Deadlock

Poiché i thread worker accedono concorrentemente a variabili e strutture condivise (la tabella delle stanze, lo stato delle scacchiere, i socket dei giocatori), la sincronizzazione è indispensabile per prevenire **Race Conditions**.

Granularità dei Mutex Adottata:

1. **Mutex Globale della Lobby (g_lobby_mutex):** inizializzato staticamente tramite la macro PTHREAD_MUTEX_INITIALIZER. Protegge l'accesso all'elenco dei client connessi e la creazione/ricerca delle stanze di gioco.
2. **Mutex Dinamico di Partita (Game.lock):** allocato in ogni struttura di match e inizializzato dinamicamente con pthread_mutex_init(). La granularità fine garantisce che le operazioni interne a una partita (es. verifica di un colpo) non blocchino i thread che operano su altre partite indipendenti.
3. **Mutex di Scrittura su Socket (ClientSession.send_lock):** le socket TCP sono full-duplex, consentendo lettura e scrittura simultanee. Tuttavia, se due thread tentano di scrivere contemporaneamente sullo stesso socket client (ad esempio durante una notifica asincrona di turno o di colpo), i flussi di byte potrebbero sovrapporsi. Il lock send_lock serializza le chiamate a send_line() rendendo atomico ogni messaggio trasmesso.

Risoluzione Formale del Deadlock:

Per prevenire il deadlock è stata definita una gerarchia rigida e unidirezionale di acquisizione dei lock:

• Se un'operazione richiede sia il lock della Lobby che il lock di una Partita, viene acquisito **sempre per primo g_lobby_mutex** e solo successivamente Game.lock.

• Il rilascio avviene rigorosamente nell'ordine inverso.

• Non esiste alcun punto nel programma in cui un thread tenta di acquisire il lock della lobby mentre detiene già il lock di una partita, escludendo matematicamente ogni possibilità di attesa circolare.

5. Multiplazione dell'I/O nel Client: System Call `select()`

Nel Client, quando un giocatore crea una partita, il programma deve rimanere in attesa che uno sfidante si colleghi (evento proveniente dal socket di rete), ma deve anche consentire all'utente di digitare un comando da tastiera (es. annullare la stanza digitando cancel o 0).

Se si utilizzasse una normale chiamata di lettura bloccante come fgets() o read(STDIN_FILENO), il processo rimarrebbe congelato in attesa della tastiera, impossibilitato a ricevere le notifiche di rete in arrivo dal server. Per monitorare contemporaneamente più descrittori si impiega la system call POSIX **select()**:

Primitiva POSIX	Ruolo nel Client
`fd_set readfds;`	Insieme dei descrittori di file da controllare in lettura.
`FD_ZERO(&readfds;);`	Azzera l'insieme dei descrittori all'inizio di ogni iterazione.
`FD_SET(STDIN_FILENO, &readfds;);` `FD_SET(sock, &readfds;);`	Inserisce sia la tastiera (file descriptor 0) che il socket di rete nell'insieme da monitorare.
`select(maxfd + 1, &readfds;, ...);`	Blocca il processo fino a quando almeno uno dei due canali è pronto per la lettura.
`FD_ISSET(fd, &readfds;)`	Verifica quale descrittore ha generato l'evento ed esegue l'azione opportuna.

Questa soluzione evita l'overhead di dover creare un secondo thread nel client, garantendo semplicità e massima reattività.

6. Trattamento dei Segnali Asincroni (`SIGPIPE` e `SIGINT`)

In ambiente UNIX la gestione dei segnali è indispensabile per garantire la robustezza delle applicazioni di rete.

Il Rischio di SIGPIPE e la sua Neutralizzazione:
Nelle comunicazioni via socket TCP, se un processo tenta di scrivere su un descrittore il cui estremo remoto è stato chiuso improvvisamente (ad esempio per arresto forzato del client o perdita di connettività), il kernel genera il segnale SIGPIPE. L'azione predefinita di SIGPIPE in ambiente UNIX è la terminazione immediata del processo.
Per proteggere il server:
1. All'avvio del main viene impostato: signal(SIGPIPE, SIG_IGN);.
2. Nelle chiamate di invio viene specificato il flag MSG_NOSIGNAL in send().
La scrittura fallita restituisce semplicemente -1 con errno == EPIPE, consentendo al thread worker di intercettare l'errore e procedere alla rimozione pulita del client senza alcun crash del server.

Gestione di SIGINT (Ctrl+C):
Sia nel server che nel client viene registrata una funzione di gestione (sigaction) per SIGINT. Alla pressione di Ctrl+C, il signal handler imposta una variabile di stato atomica (g_server_running = 0) e chiude i socket di ascolto, consentendo al processo di deallocare ordinatamente le strutture in memoria e terminare con stato 0.

7. Robustezza e Gestione Difensiva dell'Input (Hardening Anti-Break)

Il software è stato ingegnerizzato con un approccio **completamente difensivo** per resistere a test di rottura, fuzzing e valori fuori limite.

1. Eliminazione delle Vulnerabilità da Terminale (`safe_read_line`):
Le funzioni canoniche come scanf("%d") o gets() presentano vulnerabilità critiche: se l'utente digita caratteri non numerici (es. abc), scanf fallisce senza rimuovere i caratteri da stdin, innescando loop infiniti di errore. È stata pertanto sviluppata la funzione safe_read_line():
• Legge da terminale tramite fgets() su buffer fisso;
• Se l'utente digita una riga più lunga del buffer, un ciclo iterativo su getchar() **scarica e distrugge tutti i caratteri eccedenti su stdin** fino al newline, impedendo che 'sporchino' le letture successive;
• La conversione numerica è affidata a strtol() con verifica di endptr ed errno: qualsiasi carattere non numerico, numero negativo o valore fuori range viene rilevato e rifiutato con un messaggio d'errore chiaro.

2. Protezione del Server da Fuzzing e Buffer Poisoning (`recv_line`):
Anche sulla rete un interlocutore malevolo (es. script automatizzato o netcat) potrebbe trasmettere righe prive di newline lunghe migliaia di byte. La funzione di ricezione recv_line() legge carattere per carattere fino a \n: se la riga supera la dimensione massima consentita, i byte in eccesso vengono drenati e scartati. In tal modo il server non subisce alcun buffer overflow e lo stream TCP resta sincronizzato per il comando successivo.

3. Macchina a Stati Finiti Zero-Trust:
Il server non confida mai nello stato del client. Ciascun comando è valido esclusivamente se il client si trova nello stato FSM appropriato (ad es. un colpo FIRE viene categoricamente respinto se il client si trova nella lobby o se non è il suo turno).

8. Logica di Gioco e Modulo Scacchiera

Le meccaniche della Battaglia Navale sono incapsulate nel modulo board.c, garantendo netta separazione tra networking e logica applicativa:

• **Griglia di Gioco:** matrice 10x10 rappresentata dall'enumerazione CellState (CELL_EMPTY, CELL_SHIP, CELL_MISS, CELL_HIT, CELL_SUNK).

• **Composizione della Flotta (5 navi):** 1 Portaerei (5 caselle), 1 Corazzata (4 caselle), 1 Incrociatore (3 caselle), 1 Sottomarino (3 caselle), 1 Cacciatorpediniere (2 caselle).

• **Validazione del Piazzamento:** la funzione board_can_place_ship() accerta che le coordinate siano comprese tra 0 e 9, che la nave non superi i bordi e che non vi siano sovrapposizioni. Il sistema supporta l'estensione della nave in tutte e quattro le direzioni cardinali (Destra/Sinistra per orientamento orizzontale, Basso/Alto per quello verticale), proponendo la scelta all'utente quando entrambe le direzioni sono ammissibili.

• **Risoluzione dei Colpi e Determinazione Vittoria:** la funzione board_receive_shot() aggiorna lo stato della cella. Se una nave subisce colpi pari alla sua dimensione, viene marcata come affondata (CELL_SUNK) e viene decrementato il contatore delle navi vive. Tutte le celle dell'unità affondata vengono convertite nello stato CELL_SUNK su entrambi i client (radar di tiro e flotta alleata), offrendo un riscontro grafico univoco ed evitando qualsiasi ambiguità con colpi isolati su altre unità. Quando ships_alive == 0, il server dichiara la fine della partita.

9. Gestione dei Casi Limite e Requisiti Opzionali

Gestione della Disconnessione dell'Avversario (Requisito Opzionale pag. 3):

La traccia d'esame riporta a pagina 3 il requisito opzionale: *«se un giocatore cade, l'altro giocatore può decidere di terminare la partita o aspettare che un altro giocatore si unisca»*. Questa funzionalità è stata implementata come segue:

1. Nel momento in cui un client interrompe la connessione (rilevata da recv_line() <= 0 o errore di socket), il thread worker invoca lobby_remove_client().

2. Se la partita era in corso, il giocatore rimasto riceve immediatamente la notifica speciale OPPONENT_DISCONNECTED e transita nello stato STATE_OPPONENT_LEFT.

3. Al giocatore superstite viene mostrato un menu decisionale interattivo:
- **Opzione 1 (Vittoria a tavolino):** la partita viene chiusa con vittoria per abbandono e l'utente torna alla lobby principale; in questo caso la richiesta di rivincita viene inibita, non essendovi più un avversario connesso.
- **Opzione 2 (Attesa nuovo giocatore):** la stanza viene resettata e rimessa nello stato di attesa (STATE_WAITING_OPPONENT), permettendo a un nuovo utente collegato di unirsi alla partita.

Meccanismo di Rivincita (Rematch) con Multiplexing I/O:

Al termine della sfida, entrambi i giocatori possono esprimere la preferenza di disputare una nuova partita. L'interfaccia client sfrutta la system call select() per monitorare sia la tastiera che la socket: se un giocatore propone la rivincita per primo, il server trasmette la notifica REMATCH_OFFERED che appare istantaneamente sullo schermo dell'avversario. Se entrambi accettano (REMATCH 1), il server azzera le scacchiere e riavvia la fase di piazzamento; qualora uno dei due rifiuti (REMATCH 0), entrambi vengono reindirizzati alla lobby principale in totale coerenza di stato.

10. Conclusioni e Verifica Sperimentale

Il progetto è stato collaudato sia manualmente su più terminali che mediante uno script automatizzato di stress-testing e fuzzing. I risultati hanno confermato:

- **Compilazione a zero warning** con i flag più severi del compilatore: gcc -Wall -Wextra -Wpedantic -std=c11.
- **Assenza di Memory Leak e Corruzioni:** collaudato con successo con AddressSanitizer (target make debug con -fsanitize=address,undefined).
- **Comportamento Deterministico:** perfetta coordinazione dei turni e immunità totale a disconnessioni forzate o input errati.

L'architettura sviluppata soddisfa integralmente tutte le specifiche della traccia, dimostrando piena padronanza della programmazione concorrente e dei costrutti di sistema operativi in ambiente UNIX/Linux.